Bicol University Polangui, Albay
Web System and Technology
2nd Semester S.Y 2025-2026

IT 112 WEB SYSTEMS AND TECHNOLOGIES

BU TRANSAKTO

FINAL PROJECT PROPOSAL

GROUP NAME: KMJ's

MEMBERS:

BELLEN, JUVY

HERMO, MA. MERA

INTIA, KIMBERLY ANNE

REALUBIT, JANET

SEPTO, JESSECA

COURSE & BLOCK:

BSIT-2A

I. Screenshot of Working Edit/Delete Features

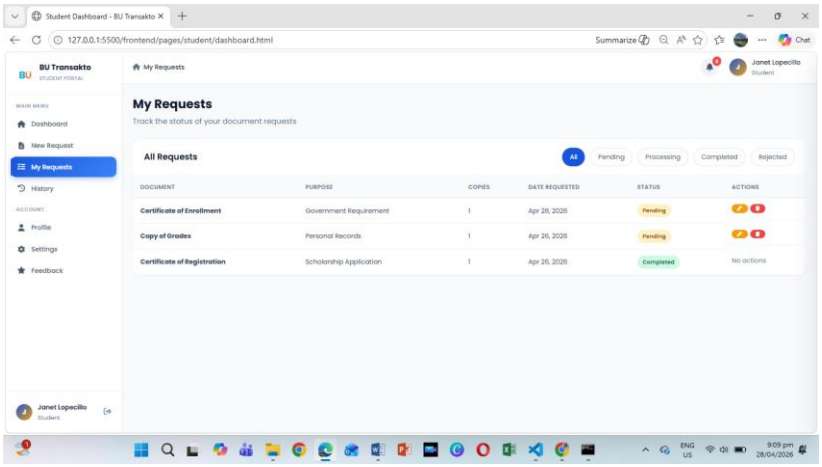


Figure 1: Student's pending requests with Edit/Delete actions.

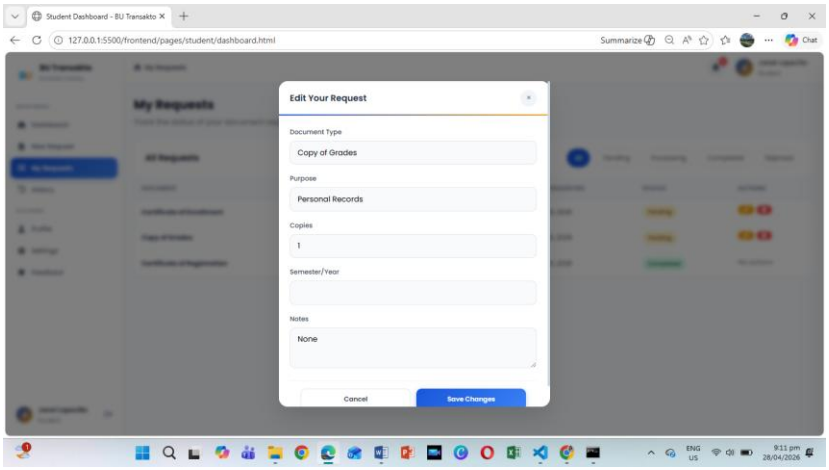


Figure 2: Edit request modal pre-filled with current request details (document type, purpose, copies, semester, notes).

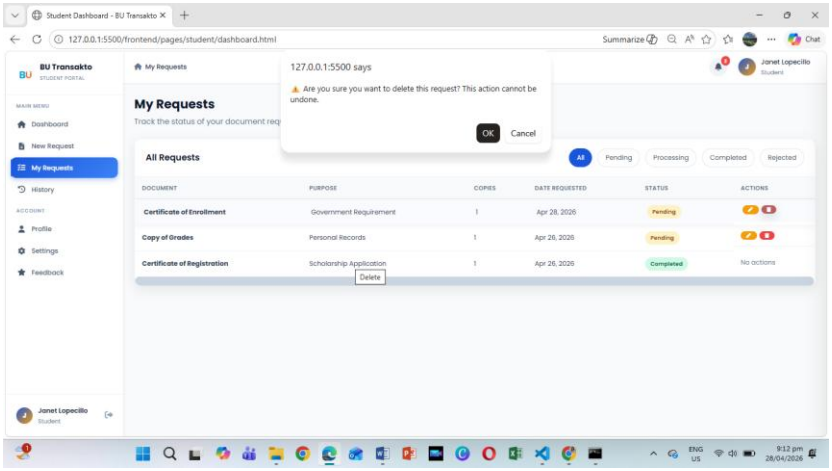


Figure 3: Delete confirmation prompt asking "Are you sure you want to delete this request? This action cannot be undone."

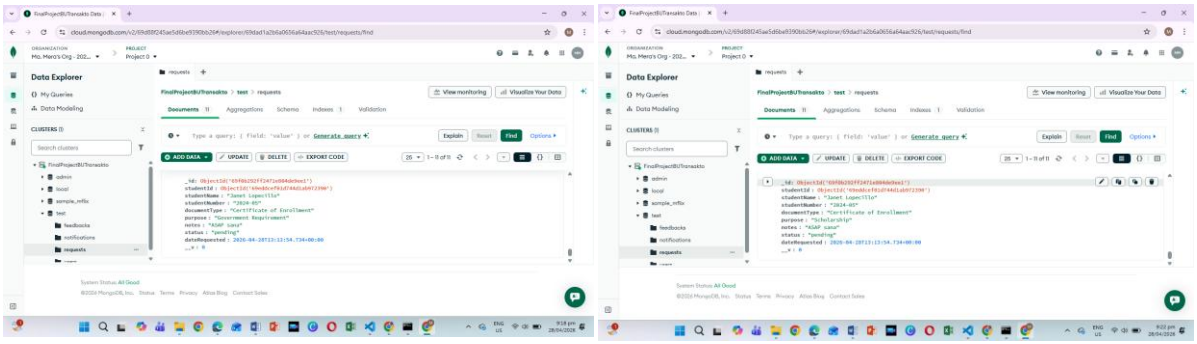


Figure 5: MongoDB Atlas shows the updated fields (e.g., purpose changed from "Government Requirement" to "Scholarship").

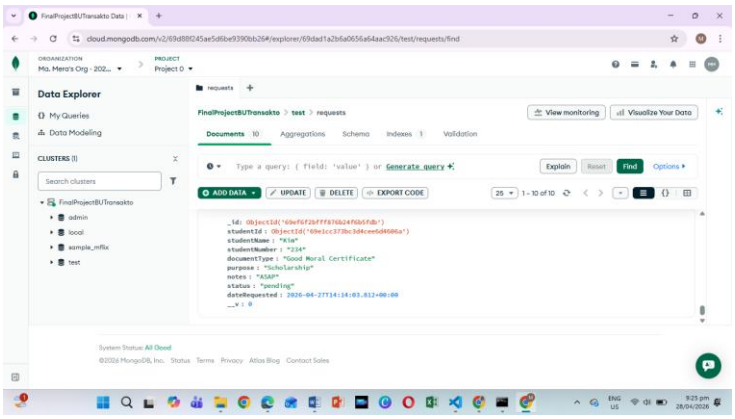


Figure 6: The deleted request is removed from the requests collection. From 11 requests to 10 requests.

II. JavaScript Snippets (Frontend)

<u>Student Dashboard – Render Row with Edit/Delete (only for pending)</u>

From student-dashboard.js – renderFullRequestRow:

```
    static renderFullRequestRow(request) {
    const status = Utils.getStatusColor(request.status);
    const isPending = request.status === 'pending';

    return `
        <tr>
            <td><strong>${Utils.sanitize(request.documentType)}</strong></td>
            <td>${Utils.sanitize(request.purpose || '-')}</td>
            <td>${request.copies || 1}</td>
            <td>${Utils.formatDate(request.dateRequested, 'short')}</td>
            <td><span class="status-badge status-${request.status}">${status.label}</span></td>
            <td class="action-btns">
                ${isPending ? `
                    <button class="btn btn-warning btn-xs" onclick="StudentDashboard.openEditRequestModal(
                        <i class="fas fa-pen"></i>
                    </button>
                    <button class="btn btn-danger btn-xs" onclick="StudentDashboard.confirmDeleteRequest('
                        <i class="fas fa-trash"></i>
                    </button>
                ` : '<span class="text-muted">No actions</span>'}
            </td>
        </tr>`;
}
```

Save Edited Request (PUT)

```javascript
    static async saveEditedRequest() {
        const id = document.getElementById('editRequestId').value;
        const updatedData = {
            documentType: document.getElementById('editDocType').value,
            purpose: document.getElementById('editPurpose').value,
            copies: parseInt(document.getElementById('editCopies').value),
            semesterYear: document.getElementById('editSemesterYear').value,
            notes: document.getElementById('editNotes').value
        };

        try {
            Utils.showLoading('Updating request...');
            await api.updateRequest(id, updatedData);
            Utils.hideLoading();
            Utils.showToast('Request updated successfully', 'success');
            this.closeEditModal();
            await this.refreshAllData();
            this.loadMyRequests();
        } catch (error) {
            Utils.hideLoading();
            Utils.showToast(error.message || 'Update failed', 'error');
        }
    }
```

Delete Request with Confirmation

```javascript
    static async confirmDeleteRequest(requestId) {
        const request = this.requests.find(r => r._id === requestId);
        if (!request || request.status !== 'pending') {
            Utils.showToast('You can only delete pending requests', 'warning');
            return;
        }
        if (!confirm('⚠️ Are you sure you want to delete this request? This action cannot be undone.')) r
        try {
            Utils.showLoading('Deleting...');
            await api.deleteRequest(requestId);
            Utils.hideLoading();
            Utils.showToast('Request deleted', 'success');
            await this.refreshAllData();
            this.loadMyRequests();
        } catch (error) {
            Utils.hideLoading();
            Utils.showToast(error.message || 'Delete failed', 'error');
        }
    }
```

III. Backend Snippets (Routes & Controllers)

Request Routes with Permission Logic (requestRoutes.js)

```javascript
// Update a request - owner student (if pending) OR staff
router.put('/:id', protect, async (req, res) => {
  try {
    const request = await Request.findById(req.params.id);
    if (!request) return res.status(404).json({ message: 'Request not found' });

    const isOwner = request.studentId.toString() === req.user._id.toString();
    const isStaffUser = req.user.role === 'staff';

    // Authorization
    if (!isOwner && !isStaffUser) {
      return res.status(403).json({ message: 'Not authorized to edit this request' });
    }
    if (isOwner && request.status !== 'pending') {
      return res.status(400).json({ message: 'Cannot edit a request that is already ' + request.status });
    }

    const allowedUpdates = ['documentType', 'purpose', 'notes', 'copies', 'semesterYear'];
    const updates = {};
    for (let key of allowedUpdates) {
      if (req.body[key] !== undefined) updates[key] = req.body[key];
    }
    const updatedRequest = await Request.findByIdAndUpdate(req.params.id, updates, { new: true, runValidators: true });
    res.json(updatedRequest);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

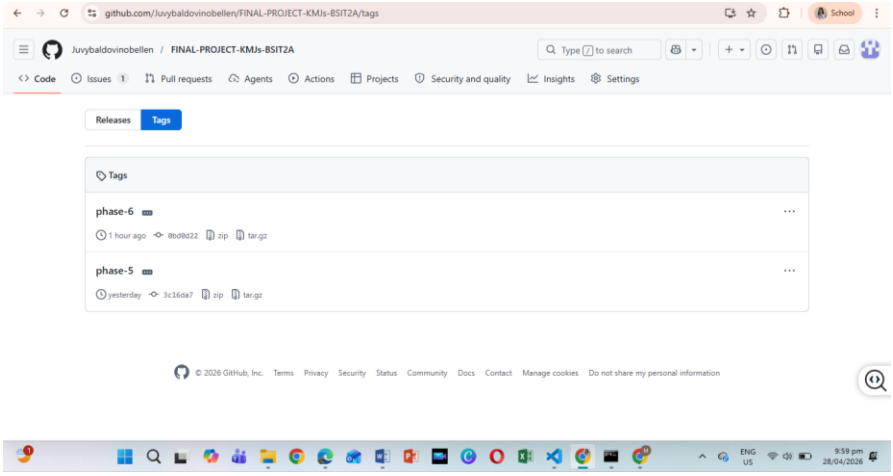
```
// Delete a request - owner student (if pending) OR staff
router.delete('/:id', protect, async (req, res) => {
  try {
    const request = await Request.findById(req.params.id);
    if (!request) return res.status(404).json({ message: 'Request not found' });

    const isOwner = request.studentId.toString() === req.user._id.toString();
    const isStaffUser = req.user.role === 'staff';

    if (!isOwner && !isStaffUser) {
      return res.status(403).json({ message: 'Not authorized to delete this request' });
    }
    if (isOwner && request.status !== 'pending') {
      return res.status(400).json({ message: 'Cannot delete a request that is already ' + request.status });
    }

    await Request.findByIdAndDelete(req.params.id);
    res.json({ message: 'Request deleted successfully' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

IV. API Methods Added to api.js

```
    async updateRequestStatus(requestId, status) {
        return await this.request(`/requests/${requestId}/status`, 'PUT', { status });
    }

        // Full update of a request (staff only)
    async updateRequest(requestId, data) {
        return await this.request(`/requests/${requestId}`, 'PUT', data);
    }

    // Delete a request (staff only)
    async deleteRequest(requestId) {
        return await this.request(`/requests/${requestId}`, 'DELETE');
    }

    // Delete a user (staff only)
    async deleteUser(userId) {
        return await this.request(`/users/${userId}`, 'DELETE');
    }
```

V. GitHub Commit Logs

0bd0d22 (tag: phase-6) feat(phase6): complete CRUD for student requests



VI. Reflections from each members

Frontend Developer (Kimberly Anne Intia)

As a Frontend Developer, working on adding edit and delete UI features, creating modals/forms, and binding events to buttons improved my understanding of user interaction and interface functionality. I learned how to design interactive elements that make the system more user-friendly and efficient. This task also strengthened my JavaScript skills, particularly in handling events and connecting UI actions to system functions.

Backend Developer (Janet L. Realubit)

As a Backend Developer, implementing PUT and DELETE endpoints and updating controllers enhanced my knowledge of handling data modification and deletion within the system. I learned the importance of properly managing requests to ensure updates and deletions are processed accurately. This responsibility improved my problem-solving skills and reinforced the role of backend logic in maintaining system reliability.

Github Manager (Juvy Bellen)

As a GitHub Manager, handling branch merging and commit synchronization taught me the importance of organized collaboration and version control. I learned how to manage code updates from different team members while ensuring changes are integrated smoothly. This task strengthened my ability to maintain a stable repository and support efficient team workflows.

Database Manager (Ma. Mera Hermo)

As a Database Manager, verifying database document updates and deletions helped me understand the importance of maintaining data accuracy and consistency. I learned how to monitor database changes and ensure that all operations are correctly reflected in stored records. This role improved my attention to detail and emphasized the significance of proper database management in system performance.

Documentation Officer (Jessseca Mae Septo)

As a Documentation Officer, preparing visuals and notes for the lab report helped me improve my ability to organize and communicate technical information clearly. I learned how to present

workflows, outputs, and important observations in a structured format. This role highlighted the importance of proper documentation in making project results easy to understand and review.